Sri Raghavendra Educational Institutions Society(R)

Sri Krishna Institute of Technology

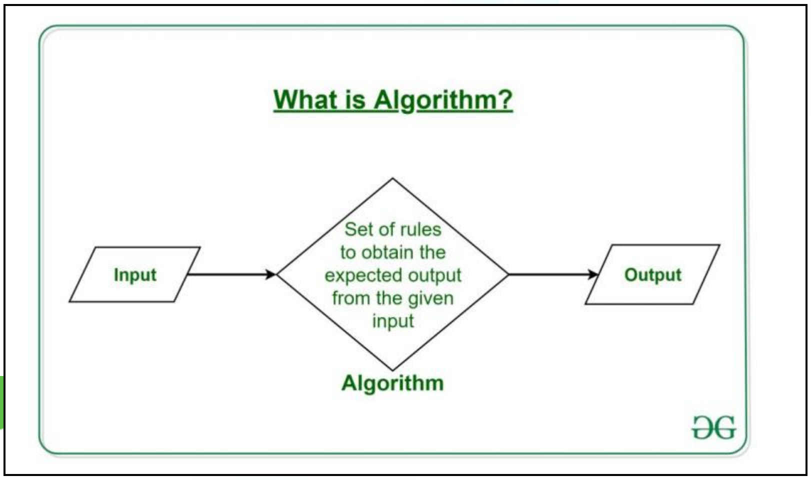
(Accredited by NAAC Approved by A.I.C.T.E. New Delhi, Recognized byGovt. of Karnataka
Affiliated to V.T U., Belagavi)

#29, Chimney Hills, Hesaraghatta Main Road, Chikkabanavara Post, Bangalore- 560090

LABORATORY MANUAL

Analysis and Design of Algorithms Lab

[BCSL404]



Compiled by

Mrs. Ragini Krishna,

Asst. Professor

Name of the Student:	
USN:	
Branch/Semester:	
Academic Year:	

GENERAL INSTRUCTIONS

The resource has been established as per requirement on need basis. In order to utilize the resource effectively and efficiently we need to follow few guidelines.

Do's

- Students **must** leave your footwear outside the lab at designated place.
- Students **must** keep your belongings in the designated place.
- Students **must** maintain discipline & follow ethics in the lab.
- Students **must** indicate the Login & Logout time on the record.
- Students **must** come prepare with Observation / Data sheet.
- Students **must** enter the lab and login to the allotted computer.
- Students **must** use the computer lab for Academic related activities.
- Students **must** use the computer lab for Academic related activities.
- Students **must** keep their working area orderly & neatly.
- Students **must** turn off the computer and keep the chairs properly before leaving the lab.

Don'ts

- Students **must not** eat food, chew gum in the lab.
- Students **must not** change the existing setup in the lab.
- Students **must not** disrupt, or trouble other students in the lab.
- Students **must not** interfere or tamper anyone else's work.
- Students **must not** access any computer or data without permission.
- Students **must not** connect the external storage to avoid transfer of virus.

SL. NO.		Page No.

1.	Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Kruskal's algorithm.	13
2.	Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Prim's algorithm.	
3.	a. Design and implement C/C++ Program to solve All-Pairs Shortest Paths problem using Floyd's algorithm. b. Design and implement C/C++ Program to find the transitive closure using Warshal's algorithm.	
4.	Design and implement C/C++ Program to find shortest paths from a given vertex in a weighted connected graph to other vertices using Dijkstra's algorithm.	
5.	Design and implement C/C++ Program to obtain the Topological ordering of vertices in a given digraph.	
6.	Design and implement C/C++ Program to solve 0/1 Knapsack problem using Dynamic Programming method.	
7.	Design and implement C/C++ Program to solve discrete Knapsack and continuous Knapsack problems using greedy approximation method.	
8.	Design and implement C/C++ Program to find a subset of a given set S = {sl , s2,.....,sn} of n positive integers whose sum is equal to a given positive integer d.	
9.	Design and implement C/C++ Program to sort a given set of n integer elements using Selection Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.	
10.	Design and implement C/C++ Program to sort a given set of n integer elements using Quick Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.	
11.	Design and implement C/C++ Program to sort a given set of n integer elements using Merge Sort method and compute its time complexity. Run the program for varied values of n> 5000, and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator method.	
12.	Design and implement C/C++ Program for N Queen's problem using Backtracking.	

Sri Raghavendra Educational Institutions Society(R)

Sri Krishna Institute of Technology

(Accredited by NAAC Approved by A.I.C.T.E. New Delhi, Recognized byGovt. of Karnataka
Affiliated to V.T U., Belagavi)

#29, Chimney Hills, Hesaraghatta Main Road, Chikkabanavara Post, Bangalore- 560090

Department of Information Science and Engineering

Vision

To nurture students to be globally competent professionals with a strong software development expertise, a passion for research, and a commitment to ethical values.

Mission

Our Mission is to:

1. Produce technically competent graduates through a well structured curriculum and interdisciplinary applications.

2. Create a competitive ambience for grooming young minds to meet the industry trends and become self-sustainable.

3. Impart knowledge of innovative technologies to encourage research and higher studies within the core areas of computation.

Sri Raghavendra Educational Institutions Society(R)

Sri Krishna Institute of Technology

(Accredited by NAAC Approved by A.I.C.T.E. New Delhi, Recognized byGovt. of Karnataka Affiliated to V.T U., Belagavi)

#29, Chimney Hills, Hesaraghatta Main Road, Chikkabanavara Post, Bangalore- 560090

Vision of the Institute

To create a community of knowledgeable and competent engineers to embody global standards of excellence and drive innovation and progress in industries, businesses, and research organizations around the world.

Mission of the Institute

To facilitate an inclusive and supportive learning environment that fosters collaboration, creativity, and the pursuit of excellence in engineering.

Program Educational Objectives (PEOs)

PEO 1: Work effectively as an individual and in a team, exhibiting leadership qualities to meet the goals of the organization.

PEO2: To be able to ad pt to the evolving technical challenges and changing career opportunities and to pursue higher education.

PEO3: Learn to apply modern skills, techniques, and engineering tools to create computational systems.

Program Specific Objectives (PSOs)

PSO1: To understand and process the principles of mathematics in the field of information Science by applying different design principles.

PSO2: To impart the knowledge by experimental methods in multidisciplinary domains.

PSO3: To inculcate communication skills and team work in developing sustainable software's by imparting professional and ethical values.

Program Outcomes (POs)

1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice

7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

1. Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Kruskal's algorithm.

ALGORITHM Kruskal(G)

//Kruskal's algorithm for constructing a minimum spanning tree
//**Input**: A weighted connected graph G = (V, E)
//**Output**: E_T, the set of edges composing a minimum spanning tree of G sort E in Non-decreasing order of the edge weights $w(e_{i1}) \leq \ldots \leq w(e_{i|E|})$

$E_T \leftarrow \Phi$; ecounter $\leftarrow$ 0 //initialize the set of tree edges and its size
k $\leftarrow$ 0 //initialize the number of processed edges
while ecounter < |V| - 1 **do**
    k $\leftarrow$ k + 1
    if E_T U [e_{ik}} is acyclic
        $E_T \leftarrow E_T$ U {e_{ik}); ecounter $\leftarrow$ ecounter + 1
return E_T

<u>Program</u>

```
#include<stdio.h>
#include<conio.h>
int min=0,cost[10][10],parent[10],i,j,x,y,n;
void main()
{
        int count=0,tot=0,flag=0;
        void find_min();
        int check_cycle();
        clrscr();
        printf("enter the no of vertices\n");
        scanf("%d",&n);
        printf("enter the matrix\n");
        printf("enter 999 for self loops and no edge\n");
        for(i=1;i<=n;i++)
        for(j=1;j<=n;j++)
        {
                scanf("%d",&cost[i][j]);
                parent[j]=0;
        }
        while(count!=n-1&&min!=999)
```

```
	{
		find_min();
		flag=check_cycle(x,y);
		if(flag==1)
		{
			printf("%Fd-->%d=%d\n",x,y,cost[x][y]);
			count++;
			tot+=cost[x][y];
		}
		cost[x][y]=cost[y][x]=999;
	}
	printf("the total cost=%d",tot);

}

void find_min()
{
	min=999;
	for(i=1;i<=n;i++)
	for(j=1;j<=n;j++)
	if(cost[i][j]<min)
	{
		min=cost[i][j];
		x=i;
		y=j;
	}
}
int check_cycle(int x,int y)
{
	if((parent[x]==parent[y])&&(parent[x]!=0))
		return 0;
	else if(parent[x]==0&&parent[y]==0)
		parent[x]=parent[y]=x;
	else if(parent[x]==0)
		parent[y]=parent[x];
	else if(parent[y]==0)
		parent[y]=parent[x];
	else if(parent[x]!=parent[y])
		parent[y]=parent[x];
	return 1;
}
```

Output

```
enter the no of vertices
4
enter the matrix
enter 999 for self loops and no edge
999 1 5 2
1 999 999 999
5 999 999 3
2 999 3 999
1-->2=1
1-->4=2
3-->4=3
the total cost=6
```

2. Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Prim's algorithm.

ALGORITHM Prim(G)

//Prim's algorithm for constructing a minimum spanning tree
//**Input**: A weighted connected graph G = (V, E)
//**Output**: Er, the set of edges composing a minimum spanning tree of G

$V_T \leftarrow$ { v_0] //the set of tree vertices can be initialized with any vertex
$E_T \leftarrow \Phi$
for i $\leftarrow$ 1 to |V| - 1 **do**
find a minimum-weight edge e* = (v*, u*) among all the edges (v, u) such that v is
in V_T and u is in V - V_T
$V_T \leftarrow V_T$ U {u*]
$E_T \leftarrow E_T$ U {e*]
return E_T

Program

```
#include<stdio.h>
#include<conio.h>
int n,c[20][20],i,j,visited[20];
void prim();
void main()
{
	printf("enter the no of vertices\n");
	scanf("%d",&n);
	printf("enter the cost matrix\n");
	for(i=1;i<=n;i++)
	{
		for(j=1;j<=n;j++)
			scanf("%d",&c[i][j]);
			visited[i]=0;
	}
	prim();
}
void prim()
{
	int min,b,a,count=0,cost=0;
	min=999;
```

```
        visited[1]=1;
        while(count<n-1)
        {
                min=999;
                for(i=1;i<=n;i++)
                        for(j=1;j<=n;j++)
                                if(visited[i]&&!visited[j]&&min>c[i][j])
                                {
                                        min=c[i][j];
                                        a=i;
                                        b=j;
                                }
                printf("%d->%d=%d\n",a,b,c[a][b]);
                cost+=c[a][b];
                visited[b]=1;
                count++;
        }
        printf("\n total cost of spanning tree is %d",cost);
}
```

Output

```
enter the no of vertices
4
enter the cost matrix
999 1 5 2
1 999 999 999
5 999 999 3
2 999 3 999
1->2=1
1->4=2
4->3=3

 total cost of spanning tree is 6
```

3. a. Design and implement C/C++ Program to solve All-Pairs Shortest Paths problem using Floyd's algorithm.
b. Design and implement C/C++ Program to find the transitive closure using Warshal's algorithm.

a) ALGORITHM Floyd(W[l..n, l..n])

//Implements Floyd's algorithm for the all-pairs shortest -paths problem
//**Input**: The weight matrix W of a graph with no negative-length cycle
//**Output**: The distance matrix of the shortest paths' lengths

```
D ← W                    //is not necessary if W can be overwritten
 for k ← 1 to n do
   for i ← 1 to n do
      for j ← 1 to n do
             D[i. j] ← min{D[i, j], D[i, k] + D[k, j])
 retnrn D
```

Program

```
#include<stdio.h>
#include<conio.h>
#include<omp.h>
#define INFINITY 999

int min(int a,int b)
{
  return a<b?a:b;
}

void floyd(int w[10][10],int n)
{
        int i,j,k;

        for(k=1;k<=n;k++)
                for(i=1;i<=n;i++)
                        for(j=1;j<=n;j++)
                                w[i][j]=min(w[i][j],w[i][k]+w[k][j]);
}
```

```
int main(void)
{
  int a[10][10],n,i,j;

  printf("\n Enter the no. of vertices:");
  scanf("%d",&n);
  printf("\n Enter the cost matrix, 0-self loop and 999-no edge\n");
  for(i=1;i<=n;i++)
    for(j=1;j<=n;j++)
        scanf("%d",&a[i][j]);

 floyd(a,n);

  printf("\n Shortest path matrix:\n");
  for(i=1;i<=n;i++)
  {
     for(j=1;j<=n;j++)
         printf("%d\t",a[i][j]);
         printf("\n");
  }


  return 0;
}
```

b) ALGORITHM Warshall(A[l..n, l..n])

//Implements Warshall's algorithm for computing the transitive closure
//Input: The adjacency matrix *A* of a digraph with *n* vertices
//Output: The transitive closure of the digraph

$R^{(O)} \leftarrow A$
for k $\leftarrow$ 1 to n **do**
  for i $\leftarrow$ 1 to n **do**
    for j $\leftarrow$ 1 to n **do**
      $R^{(k)}[i, j] \leftarrow R^{(k-l)}[i, j]$ **or** $(R^{(k-l)}[i, k]$ and $R^{(k-l)}[k, j])$

return $R^{(n)}$

Program

```
#include<stdio.h>
#include<conio.h>
void main()
{
	int n,i,j,k,a[20][20],t[20][20];
	printf("enter the no of vertices");
	scanf("%d",&n);
	printf("enter the adjacency matrix\n");
	for(i=1;i<=n;i++)
	for(j=1;j<=n;j++)
	{
		scanf("%d",&a[i][j]);
		t[i][j]=a[i][j];
	}
	for(k=1;k<=n;k++)
	for(i=1;i<=n;i++)
	for(j=1;j<=n;j++)
	t[i][j]=t[i][j]||t[i][k]&&t[k][j];
	printf("the transitive closure for a directed graph is\n");
	for(i=1;i<=n;i++)
	{
		for(j=1;j<=n;j++)
			printf("%d",t[i][j]);
			printf("\n");
	}

}
```

Output

```
enter the no of vertices
4
enter the adjacency matrix
0 0 1 0
1 0 0 0
0 1 0 1
1 0 0 0
the transitive closure for a directed graph is
1111
1111
1111
1111
```

4. Design and implement C/C++ Program to find shortest paths from a given vertex in a weighted connected graph to other vertices using Dijkstra's algorithm.

ALGORITHM Dijkstra(G, s)

//Dijkstra's algorithm for single-source shortest paths

//**Input**: A weighted connected graph G = (V, *E)* with nonnegative weights and its vertex *s*

//**Output**: The length d_v of a shortest path from s to v and its penultimate vertex p_v for every vertex v in V

Initialize(Q) //initialize vertex priority queue to empty

for every vertex *v* in *V* **do**

$d_v + \infty$; $P_v \leftarrow$ **null**

Insert(Q, v, d_v) //initialize vertex priority in the priority queue

$d_s \leftarrow 0$; Decrease(Q, s, d_s) //update priority of s with d,

$V_T \leftarrow 0$

for i $\leftarrow$ O to |V|-1 **do**

u* $\leftarrow$ DeleteMin(Q) //delete the minimum priority element

$V_T \leftarrow V_T$ U {u*)

for every vertex u in V - V_T that is adjacent to u* do

if d_{u^*} + w(u*, u) < d_u

$d_u \leftarrow d_{u^*}$ + w(u*, u); $p_u \leftarrow$ u*

Decrease(Q, u, d_u)

Program

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int i,j,a[10][10],s[10],n,d[10],v,k,min,u;
        printf("enter the no of vertices:=>");
        scanf("%d",&n);
        printf("enter the cost matrix\n");
        printf("enter 999 for no edges between vertices\n");
        for(i=1;i<=n;i++)
        for(j=1;j<=n;j++)
                scanf("%d",&a[i][j]);
        printf("enter the source vertex\n");
        scanf("%d",&v);
        for(i=1;i<=n;i++)
```

```
        {
                s[i]=0;
                d[i]=a[v][i];
        }
        d[v]=0;
        s[v]=1;
        for(k=2;k<=n;k++)
        {
                min=999;
                for(i=1;i<=n;i++)
                if(s[i]==0&&d[i]<min)
                {
                        min=d[i];
                        u=i;
                }
                s[u]=i;
                for(i=1;i<=n;i++)
                        if(s[i]==0)
                        {
                                if(d[i]>d[u]+a[u][i])
                                        d[i]=d[u]+a[u][i];
                        }
        }
        printf("the shortest distance from %d is\n",v);
        for(i=1;i<=n;i++)
                printf("\n%d->%d=%d",v,i,d[i]);
}
```

Output

```
enter the no of vertices:=>5
enter the cost matrix
enter 999 for no edges between vertices
0 3 999 7 999
3 0 4 2 999
999 4 0 5 6
7 2 5 0 4
999 999 6 4 0
enter the source vertex
```

1
the shortest distance from 1 is

1->1=0
1->2=3
1->3=7
1->4=5
1->5=9

5. Design and implement C/C++ Program to obtain the Topological ordering of vertices in a given digraph.

Program

```
#include<stdio.h>
#include<conio.h>
void main()
{
	int a[10][10],t[10],indeg[10],n,SUM=0;
	int u,k=0,v;
	int i,j,stack[10],top=-1;
	printf("\n\n\t topological ordering \n\n");
	printf("enter the directed acyclic graph\n\n");
	printf("enter the no of vertex\t");
	scanf("%d",&n);
	printf("enter the adjacency matrix\n");
	for(i=0;i<n;i++)
	{
		for(j=0;j<n;j++)
		{
			scanf("%d",&a[i][j]);
		}
	}
	for(i=0;i<n;i++)
		indeg[i]=0;
	for(j=0;j<n;j++)
	{
		SUM=0;
		for(i=0;i<n;i++)
		{
			SUM+=a[i][j];
		}
		indeg[j]=SUM;
	}
	for(i=0;i<n;i++)
	{
		if(indeg[i]==0)
		{
			stack[++top]=i;
		}
	}
	while(top!=-1)
```

```
        {
                u=stack[top--];
                t[k++]=u;
                for(v=0;v<n;v++)
                {
                        if(a[u][v]==1)
                        {
                                indeg[v]--;
                                if(indeg[v]==0)
                                {
                                        stack[++top]=v;
                                }
                        }
                }
        }
        printf("the topological sorting list\n");
        for(i=0;i<n;i++)
        {
                printf("%d\t",t[i]+1);
        }
}
```

Output

```
 topological ordering

enter the directed acyclic graph

enter the no of vertex
5
enter the adjacency matrix
0 1 0 1 0
0 0 0 1 1
1 0 0 1 1
0 0 0 0 0
0 0 0 0 0
the topological sorting list
3	1	2	5	4
```

6. Design and implement C/C++ Program to solve 0/1 Knapsack problem using Dynamic Programming method.

ALGORITHM Knapsack(i, j)

//Implements the memory function method for the knapsack problem
//**Input**: A nonnegative integer i indicating the number of the first items being considered and a nonnegative integer j indicating the knapsack's capacity
//**Output**: The value of an optimal feasible subset of the first i items
//**Note**: Uses as global variables input arrays Weights[l .. n], V alues[l .. n], and table V[O .. n, O .. W] whose entries are initialized with -l's except for row 0 and column 0 initialized with O's

```
if V[i,j] < O
   if  j < Weights[i]
        value ← Knapsack (i - 1, j)
   else
        value ← max(Knapsack(i- 1, j),
                Values[i] + Knapsack (i - 1, j - Weights[i] ))

   V[i, j] ← value
return V[i, j]
```

Program

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
int mfk(int,int);
void sol(int,int);
int n,i,j,W,w[10],val[10],v[20][20],a[10];
void main()
{
        clrscr();
        printf("\n\n\t0/1 Knapsack using memory functions\n\n");
        printf("\nEnter the no. of items\n");
        scanf("%d",&n);
        printf("\nEnter the weights of each item\n");
        for(i=1;i<=n;i++)
                scanf("%d",&w[i]);
        printf("\nEnter the values/profits of each item\n");
        for(i=1;i<=n;i++)
                scanf("%d",&val[i]);
```

```
	printf("\nEnter the capacity of knapsack\n");
	scanf("%d",&W);
	for(i=0;i<=n;i++)
	{
		for(j=0;j<=W;j++)
		{
			if(i==0 || j==0)
				v[i][j]=0;
			else
				v[i][j]=-1;
		}
	}
	for(i=1;i<=n;i++)
		a[i]=0;
	printf("\nThe maximum possible profit is %d\n\n",k(n,W));
	getch();
}
int mfk(int i,int j)
{
	if(v[i][j]<0)
	{
		if(j<w[i])
			v[i][j]=k(i-1,j);
		else
			v[i][j]=max(k(i-1,j),k(i-1,j-w[i])+val[i]);
	}
	return v[i][j];
}

void sol(int i, int j)
{
	if(i>0)
	{
		if(v[i][j]!=v[i-1][j])
		{
			a[i]=1;
			sol(i-1,j-w[i]);
		}
		else
			sol(i-1,j);
	}
}
```

```
/*	Output

	0/1 Knapsack using memory functions

	Enter the no. of items
	4
	Enter the weights of each item
	2 1 3 2
	Enter the values/profits of each item
	12 10 20 15
	Enter the capacity of knapsack
	5
	The maximum possible profit is 37
```

7. Design and implement C/C++ Program to solve discrete Knapsack and continuous Knapsack problems using greedy approximation method

Program

```
#include<stdio.h>
int main()
{
     float weight[50],profit[50],ratio[50],Totalvalue,temp,capacity,amount;
     int n,i,j;
     printf("Enter the number of items :");
     scanf("%d",&n);
    for (i = 0; i < n; i++)
    {
       printf("Enter Weight and Profit for item[%d] :\n",i);
       scanf("%f %f", &weight[i], &profit[i]);
    }
    printf("Enter the capacity of knapsack :\n");
    scanf("%f",&capacity);

     for(i=0;i<n;i++)
         ratio[i]=profit[i]/weight[i];

    for (i = 0; i < n; i++)
      for (j = i + 1; j < n; j++)
         if (ratio[i] < ratio[j])
         {
```

```
        temp = ratio[j];
        ratio[j] = ratio[i];
        ratio[i] = temp;

        temp = weight[j];
        weight[j] = weight[i];
        weight[i] = temp;

        temp = profit[j];
        profit[j] = profit[i];
        profit[i] = temp;
      }

printf("Knapsack problems using Greedy Algorithm:\n");
for (i = 0; i < n; i++)
{
 if (weight[i] > capacity)
    break;
  else
  {
    Totalvalue = Totalvalue + profit[i];
    capacity = capacity - weight[i];
  }
}
  if (i < n)
  Totalvalue = Totalvalue + (ratio[i]*capacity);
```

```
    printf("\nThe maximum value is :%f\n",Totalvalue);
    return 0;
}
```

output:-

```
Enter the number of items :4
Enter Weight and Profit for item[0] :
2
12
Enter Weight and Profit for item[1] :
1
10
Enter Weight and Profit for item[2] :
3
20
Enter Weight and Profit for item[3] :
2
15
Enter the capacity of knapsack :
5
Knapsack problems using Greedy Algorithm:

The maximum value is :38.333332
--------------------------------
```

8. Design and implement C/C++ Program to find a subset of a given set S = {sl , s2,.....,sn} of n positive integers whose sum is equal to a given positive integer d.

Program

```
#include<stdio.h>
#include<conio.h>
int w[10],d,n,count,x[10],i,r,flag=1;
void subset(int s,int k,int r)
{
        x[k]=1;
        if(s+w[k]==d)
        {
                printf("\n\nSubset %d\n\n",++count);
                for(i=0;i<=k;i++)
                {
                        if(x[i])
                                printf("%3d",w[i]);
                }
                flag=0;
        }
        else
        {
                if(s+w[k]+w[k+1] <=d)
                        subset(s+w[k],k+1,r-w[k]);
                if((s+r-w[k]>=d) && (s+w[k+1]<=d))
                {
                        x[k]=0;
                        subset(s,k+1,r-w[k]);
                }
        }
}
void main()
{

        printf("\nEnter the no. of elements\n");
        scanf("%d",&n);
        printf("Enter the elements of set\n");
        for(i=0;i<n;i++)
        {
                scanf("%d",&w[i]);
                x[i]=0;
                r+=w[i];
```

```
        }
        printf("\nEnter the value of d\n");
        scanf("%d",&d);
        subset(0,0,r);
        if(flag)
                printf("\ndoes not match the sum to any subset\n");

}
```

```
Enter the no. of elements
5
Enter the elements of set
1
2
3
4
5

Enter the value of d
10


Subset 1

  1  2  3  4

Subset 2

  1  4  5

Subset 3

  2  3  5
```

9. Design and implement C/C++ Program to sort a given set of n integer elements using Selection Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

```
void swap(int *xp, int *yp)
{
    int temp = *xp;
    *xp = *yp;
    *yp = temp;
}

void selectionSort(int arr[], int n)
{
    int i, j, min_idx;

    // One by one move boundary of unsorted subarray
    for (i = 0; i < n-1; i++)
    {
        // Find the minimum element in unsorted array
        min_idx = i;
        for (j = i+1; j < n; j++)
          if (arr[j] < arr[min_idx])
            min_idx = j;

        // Swap the found minimum element with the first element
           if(min_idx != i)
            swap(&arr[min_idx], &arr[i]);
    }
}

/* Function to print an array */
void printArray(int arr[], int size)
{
    int i;
    for (i=0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
```

```
// Driver program to test above functions
int main()
{
        int n, i,a[20];

         printf("Ten random numbers from 0 to 99\n\n");
         for(i=0; i<10; i++)
           {
                a[i]=rand()%100;
                 printf("%d\n", a[i]);
        }
         start=clock();
        delay(1000);
        selectionSort(arr, n);

        end=clock();
         printf("the sorted array is:\n");
         for(i=0;i<9;i++)
        printf("%d\n",a[i]);
         printf("\n time taken=%f",(end-start)/CLK_TCK);

   }
```

10. Design and implement C/C++ Program to sort a given set of n integer elements using Quick Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

Algorithm

//Input: A subarray A[l,.....,r] of A[0,...,n-1]), defined by its left and right indices l and r
//Output: Subarray A[l,...,r] sorted in non-decreasing order

```
Quicksort(A[l,.....,r])
 if l<r
        S ← Partition(A[l,.....,r])
        Quicksort(A[l,.....,s-1])
        Quicksort(A[s+1,.....,r])

Partition(A[l,...,r])
        P ← A[l]
        i ← l;
        j ← r+1
        repeat
                repeat i ← i + 1 until A[i] ≥ p
                repeat j ← j - 1 until A[j] ≤ p
                swap(A[i],A[j])
        until i ≥ j
        swap(A[i],A[j])
        swap(A[l],A[j])
        return j
```

Program

```
#include<stdio.h>
#include<conio.h>
#include<dos.h>
#include<time.h>
#include<stdlib.h>
```

```
clock_t start,end;
int partition(int low,int high,int a[])
{
  int i,j,key,temp;
  i=0,j=high+1;
  key=a[low];
  while(i<=j)
  {
    do i++;while(key>=a[i]);
    do j--;while(key<a[j]);
    if(i<j)
    {
          temp=a[i];
          a[i]=a[j];
          a[j]=temp;
    }
  }
    temp=a[low];
    a[low]=a[j];
    a[j]=temp;
    return j;
}
void quick_sort(int low,int high,int a[])
{
   int mid;
   if(low<high)
   {
     mid=partition(low,high,a);
     quick_sort(low,mid-1,a);
     quick_sort(mid+1,high,a);
   }
}
void main()
{
  int n, i,a[20];

  printf("Ten random numbers from 0 to 99\n\n");
  for(i=0; i<10; i++)
  {
  a[i]=rand()%100;
  printf("%d\n", a[i]);
  }
 start=clock();
 delay(1000);
  quick_sort(0,9,a);
```

```
  end=clock();
  printf("the sorted array is:\n");
  for(i=0;i<9;i++)
  printf("%d\n",a[i]);
  printf("\n time taken=%f",(end-start)/CLK_TCK);

}

/* output
Ten random numbers from 0 to 99
46
30
82
90
56
17
95
15
48
26

the sorted array is:
15
17
26
30
46
48
56
82
90

 time taken=0.989011          */
```

11. Design and implement C/C++ Program to sort a given set of n integer elements using Merge Sort method and compute its time complexity. Run the program for varied values of n> 5000, and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

Algorithm
//**Input:** An array A[0,…..,n-1] of orderable elements.
//**Output:** Array A[0,…..,n-1] sorted in non-decreasing order.

```
Mergesort(A[0…..,n-1])
  if  n > 1
          copy A[0…..(n/2)-1] to B[0,….(n/2)-1]
          copy A[(n/2),…n-1] to C[0,….(n/2)-1]
          Mergesort(B[0,….(n/2)-1]
          Mergesort(C[0,….(n/2)-1])
          Merge(B,C,A)

Merge(B[0,…p-1), C[0,….q-1], A[0,…p+q+1])
  i ← 0; j ← 0; k ← 0
    while i < p and j < q do
      if B[i] ≤ C[j]
          A[k] ← B[i];   i ← i + 1
     else A[k] ← C[j];   j ← j + 1
     k ← k + 1
    if i = p
          copy C[j,…,q-1] to A[k,…p+q-1]
    else  copy B[i,… ,p-1] to A[k,…p+q-1]
```

Program

```
#include<stdio.h>
#include<stdlib.h>
#include<omp.h>
#include<time.h>
#define MAX 200

void simple_merge(int a[],int low,int mid,int high);
void merge_sort(int a[],int low,int high);

int main(void)
```

```
{
	int a[MAX],i=0,ch,limit;
	FILE *fi;
	char line[50];
	clock_t  startTime, endTime;
	double   time_taken;

	printf("Enter your choice\n");
	printf("1: Input from file \t 2: Random Number generation\n");
	scanf("%d",&ch);
	switch(ch)
	{	//To take input from the file
		case 1: fi = fopen("tosort.txt", "r");
				while (fgets(line,50,fi)!= NULL && i<MAX)
				{
					sscanf(line,"%3d", &limit);
					a[i] = limit;
					i++;
				}
			fclose(fi);
			break;
		//To generate randomly
		case 2: for(i=0;i<MAX;i++)
			a[i]=rand();
			break;
	}
	printf("\nBefore Sorting:\n");
	for(i=0;i<MAX;i++)
		printf("%d\t",a[i]);
	startTime= clock();
	merge_sort(a,0,MAX-1);
	endTime = clock();
	time_taken= ((double) (end - start)) / CLOCKS_PER_SEC;

	printf("Time taken is %10.9f\n", time_taken);
	printf("After Sorting:\n");
	for(i=0;i<MAX;i++)
		printf("%d\t",a[i]);
	return 0;
}

void simple_merge(int a[],int low,int mid,int high)
```

```
{
	int i=low,j=mid+1,k=low,c[MAX];

	while(i<=mid && j<=high)
	{
		if(a[i]<a[j])
		{
			c[k]=a[i];
			i++;
			k++;
		}
		else
		{
			c[k]=a[j];
			j++;
			k++;
		}
	}
	while(i<=mid)
		c[k++]=a[i++];
	while(j<=high)
		c[k++]=a[j++];

	for(i=low;i<=high;i++)
		a[i]=c[i];
}
void merge_sort(int a[],int low,int high)
{
	int mid;
	if(low<high)
	{
		mid=(low+high)/2;

		merge_sort(a,low,mid);
		merge_sort(a,mid+1,high);

		simple_merge(a,low,mid,high);
	}
}
```

12. Design and implement C/C++ Program for N Queen's problem using Backtracking.

Program

AlGORITHM Backtrack(X[l..i])

//Gives a template of a generic backtracking algorithm
//**Input**: X[Li] specifies first i promising components of a solution
//**Output**: All the tuples representing the problem's solutions

if X[Li] is a solution **write** X[l..i]
else //see Problem 8 in the exercises
for each element x є S_{i+1} consistent with X[1,…i] and the constraints **do**
X[i + 1] ← X
Backtrack(X[1..i + 1])

Program

```
//N-QUEEN'S
#include<stdio.h>
#include<conio.h>
#define max 20
int flag=1;
int place(int x[ ],int k)
{
	int i;
	for(i=0;i<k;i++)
		if(x[i]==x[k] || abs(x[i]-x[k])==abs(i-k))
			return 0;
	return 1;
}
void display(int x[ ],int n)
{
	char chessb[max][max];
	int i,j;
	for(i=0;i<n;i++)
	{
		for(j=0;j<n;j++)
		{
			chessb[i][j]='X';
```

```
				}
			}
				for(i=0;i<n;i++)
				{
					chessb[i][x[i]]='Q';

				}

				for(i=0;i<n;i++)
				{
					for(j=0;j<n;j++)
					{
						printf(" %c",chessb[i][j]);
						flag=0;
					}
					printf("\n");
				}
				printf("\n");
}
void nqueens(int n)
{
		int x[max];
		int k;
		x[0]=-1;
		k=0;
		while(k>=0)
		{
			x[k]=x[k]+1;
			while(x[k]<n&&!place(x,k))
				x[k]=x[k]+1;
			if(x[k]<n)
			{

				if(k==n-1)
					display(x,n);
				else
				{
					k++;
					x[k]=-1;
				}
			}
```

```
                    else
                              k--;
          }
}

void main()
{
          int n;

          printf("\n\n\tImplementation of NQueens problem\n\n");
          printf("\nEnter the no. of queens\n");
          scanf("%d",&n);
          printf("\nThe solution to %d Queens problem is\n\n",n);
          nqueens(n);
          if(flag)
                    printf("\n\tNo Solution to %d Queens problem\n",n);

}
```

ANALYSIS AND DESIGN OF ALGORITHMS

VIVA QUESTIONS

1. Define an algorithm?

2.Define a flowchart?

3.explain the notion of algorithm?

4.define sorting

5.define searching

7.define a graph

8.mention some of sorting techniques that comes under brute force

9.define a string and types of strings

10.define data structure
11.what are non linear data structure ? and mention

12.what are linear data structure ?and mention

13.mention some of the ways that the graph can be represented?

14.define weighted graphs

15.define path

16.define cycle

17.define directed and undirected graph

18.define simple and connected graph

19.define a tree

20.define a forest.

21.define a rooted tree

22.deffrentiate between binary tree and binary search tree

23.define sets and dictionaries

24.mention name of the algorithm design techniques or paradigm?

25. what is adt

26.differentiate between time and space efficiency

27. define basic operation

28.define worst,best,average case efficiency

29.define amortized efficiency

30.define asymptotic notation

31.using limits how do you compute order of growth?

32.what is the time complexity of linear search algorithm

33.what is the time complexity of matrix multiplication

34.mentionsome of the factors that time complexity depends on

35. what is the time complexity of bubble sort

36. what is the time complexity of selection sort

37. what is the time complexity of warshalls algorithm

38. what is the time complexity of brute force string matching.

39.what is the general plan for analyzing efficiency of recursive algorithm

40. what is the general plan for analyzing efficiency of non recursive algorithm.